

[18] An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint

저자: Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, Jiongang Ni

학회/년도: NeurIPS 2023

분량: 약 15페이지 + 부록

역할군: (C) 경쟁/비교 시스템

요약

이 논문은 벡터 검색과 속성 필터를 동시에 처리하는 효율적인 하이브리드 쿼리 프레임워크 NHQ(Native Hybrid Query)를 제안한다. 기존의 Pre-filtering (필터 먼저 적용) 또는 Post-filtering (검색 먼저 수행) 접근법은 속성 선택도가 낮거나 높을 때 극단적인 성능 저하를 보이는 반면, NHQ는 Navigable Proximity Graph(NPG) 구조를 통해 벡터 근접성과 속성 제약을 동시에 활용함으로써 최대 315배의 성능 향상을 달성한다. 특히 속성 분포의 균일성 여부에 따라 두 가지 변형(NPG-C와 NPG-A)을 제공하여, 다양한 실세계 데이터셋에서 재현율 90% 이상을 유지하면서 높은 효율성을 보장한다. NeurIPS 2023의 발표로 학술적 공신력이 높으며, 필터링된 벡터 검색 분야의 최신 기술을 대표한다.

상세분석

18.1 해결하고자 하는 문제: 하이브리드 쿼리의 비효율성

현대의 데이터 검색 애플리케이션은 단순한 벡터 유사도 검색만으로는 부족하다. 대부분의 실세계 시나리오에서는 **벡터 근접성(vector proximity)**과 **속성 제약(attribute constraints)**을 함께 처리해야 한다. 예를 들어, 이미지 검색 시스템에서는 "이 이미지와 비슷한 이미지를 찾되, 특정 카메라 모델로 촬영된 것만 원한다"는 요구가 흔하다. 문서 검색에서는 "이 텍스트와 의미가 비슷한 문서를 찾되, 특정 시간 범위와 카테고리로 제한하고 싶다"는 다중 필터가 일반적이다.

이런 **하이브리드 쿼리(HQ)** 환경에서 기존의 두 가지 접근법은 근본적인 한계를 보인다:

접근법 1: Pre-filtering (필터 우선)

1. 속성 조건을 먼저 적용: `WHERE category = 'electronics'`
2. 필터링 후 남은 데이터에 대해 ANN 검색 수행

장점:

- 검색 범위가 작아지므로, 작은 필터링 결과 집합에 대해서는 빠르다.

치명적 문제점:

- 속성 조건의 선택도가 낮으면 (예: 1% 미만의 데이터만 통과), 필터링 후 남은 데이터 양이 극소이다.
- 이 극소 데이터셋에 대해서는 벡터 인덱스(예: HNSW)가 효과적으로 작동하지 않는다. 인덱스는 일정 규모 이상의 데이터를 가정하고 설계되었기 때문이다.
- 결과적으로 순차 탐색(linear scan)으로 되돌아가야 하며, 이는 대규모 원본 데이터의 작은 부분만 건너뛰는 것이므로 오버헤드가 크다.
- 또한 별도의 부분 인덱스(partial index)를 구축해야 하므로, 저장 공간과 인덱스 유지 비용이 증가한다.

접근법 2: Post-filtering (검색 우선)

1. ANN 검색으로 top-k 결과를 먼저 구함
2. 구한 결과에서 속성 조건에 맞지 않는 항목을 제거

장점:

- 전체 데이터셋에 대해 인덱스를 최대한 활용할 수 있다.

치명적 문제점:

- 속성 조건이 까다로우면 (선택도가 낮으면), top-k 결과 중 대부분이 속성 조건에 맞지 않아 제거된다.
- 예를 들어, 벡터 유사도는 높지만 속성이 맞지 않는 데이터가 많으면, 유효한 결과를 충분히 얻으려면 k를 매우 크게 설정해야 한다 (예: 원래 k=10이지만, 실제로는 k=10,000으로 검색해야 함).
- k가 커질수록 검색 비용은 선형적으로 증가하므로, "필터 조건 때문에 비용이 폭발한다"는 역설적 상황이 발생한다.
- 예: 1% 선택도인 경우, 유효한 결과 10개를 얻으려면 벡터 검색으로 약 1,000개를 먼저 찾아야 한다.

이 두 극단적 경우 사이의 균형을 찾기 위해 NHQ가 등장한다.

18.2 핵심 아이디어: NHQ 프레임워크와 NPG 구조

NHQ의 핵심 통찰은 다음과 같다: **벡터 근접성과 속성 제약을 분리하지 말고, 인덱스 구조 자체에서 동시에 활용하자.**

Navigable Proximity Graph (NPG) — 하이브리드 인덱스 구조

NPG는 기존 근접 그래프(HNSW, NSW 등)를 확장한 구조이다:

표준 HNSW:

노드 = 벡터
간선 = 벡터 간 근접 관계

NPG:

노드 = {벡터, 속성값}
간선 = 벡터 간 근접 관계 + 속성 일관성 정보

동시 프루닝(Joint Pruning):

그래프를 탐색할 때, 후보 노드를 다음과 같이 평가한다:

candidate는 탐색 후보에 포함되는가?

← 벡터 거리가 현재 threshold 이내인가?
AND 속성이 사용자 조건을 만족하는가?

전통적 방식은 이 두 조건을 순차적으로 체크하지만, NPG의 joint pruning은:

- 벡터 거리 조건을 만족하지만 속성 조건을 만족하지 않는 방향으로의 탐색을 일찍 중단
- 속성 조건을 만족하지만 거리가 너무 먼 방향으로의 탐색을 일찍 중단

이것이 단순해 보이지만, **그래프 탐색 비용을 근본적으로 줄이는 핵심**이다.

두 가지 NPG 변형

NPG-C (Constrained) — 속성 기반 그래프 구축

아이디어: 속성 조건을 만족하는 노드들만으로 별도의 근접 그래프를 구축한다.

구축 과정:

1. 원본 데이터 D에서 속성 조건 C를 만족하는 부분집합 D' 추출
2. D'에 대해 HNSW 그래프 구축
3. 이 그래프에서만 탐색 수행

효과:

- 그래프 탐색이 항상 속성 조건을 만족하는 노드 범위 내에서만 이루어진다.
- Post-filtering의 낭비(조건 불만족 항목 버리기)가 없다.
- 그래프 크기가 작아서 탐색이 빠르다.

최적인 경우:

- 속성 분포가 **균일할 때** (예: 상품의 카테고리가 여러 개이고, 각각 충분한 데이터를 가짐)
- 속성 조건의 선택도가 **충분히 높을 때** (5% 이상)

부작용:

- 속성 분포가 **불균일할 때** (예: 특정 카테고리가 매우 드물 때), 조건을 만족하는 데이터가 극소수가 되어 NPG-C 그래프 자체의 품질(HNSW의 연결성)이 떨어진다.

NPG-A (Adaptive) — 속성 오버레이 인덱스

아이디어: 원본 근접 그래프는 유지하되, 속성 정보를 별도의 인덱스로 오버레이(overlay)한다.

구조:

- 기본 그래프: 모든 노드의 HNSW (크기: $|D|$)
- 속성 인덱스: 각 속성값에 해당하는 노드들의 목록 (비트맵, B-tree 등)

탐색:

1. HNSW 그래프에서 탐색 수행 (base 그래프는 모든 노드 포함)
2. 후보 노드를 찾을 때마다, 속성 인덱스에서 해당 속성값의 노드인지 확인
3. 속성 조건을 만족하는 후보만 취급

효과:

- 기본 그래프가 항상 완전하므로, 불균형한 속성 분포에서도 안정적이다.
- 속성 분포가 극도로 불균형할 때(예: 한 카테고리에 99% 데이터) 특히 유리하다.

부작용:

- 속성 인덱스 접근(메모리 참조) 오버헤드가 있다.
- 속성 조건 불만족 때문에 그래프 탐색 시 많은 노드가 버려질 수 있다. (Post-filtering의 부작용)

최적인 경우:

- 속성 분포가 극도로 불균형할 때
- 속성 값의 종류가 많을 때 (예: 상품 ID, 시간스탬프 등 고유성이 높은 속성)

실행 중 선택 메커니즘

논문은 NPG-C와 NPG-A 중 어느 것을 선택할지를 **데이터 통계**에 따라 결정하는 휴리스틱을 제안한다:

```
IF 속성 선택도 > threshold (예: 5%):  
    use NPG-C (constrained)  
ELSE:  
    use NPG-A (adaptive)
```

또는 **하이브리드 방식**: 두 인덱스를 모두 구축하고, 쿼리 특성에 따라 실행 시간에 선택한다.

18.3 성능 평가: 실험 결과

논문의 실험은 **10개의 실세계 데이터셋**에서 수행되었다:

데이터셋	벡터	레코드 수	속성 특성	용도
SIFT-M	128D	100M	저차원 속성	이미지 특징
GIST	960D	1M	카테고리, 레이블	이미지 검색
MSR-VTT	4096D	10K	비디오 ID, 길이	비디오 검색
OpenImages	1536D	1M	객체 카테고리	그림 검색
기타	-	-	-	-

주요 성능 지표:

1. 쿼리 지연시간(Latency)

2. Pre-filtering: 선택도 낮을 때 극악 (최악 > 1초)
3. Post-filtering: 선택도 낮을 때 극악 (k 자동 증가로 수십초)
4. NHQ (NPG-C/A): 일정하게 빠름 (< 50ms)
5. 재현율(Recall)
6. 모든 방식에서 90% 이상 유지
7. NHQ는 낮은 k값에서도 높은 recall 달성
8. 상대 속도 향상
9. Pre-filtering 대비: 최대 100배
10. Post-filtering 대비: 최대 315배
11. 평균: 50~100배

속성 선택도별 분석:

선택도 1%: NHQ가 기존 대비 100배 이상 우수
선택도 5%: NHQ가 50배 우수
선택도 10%: NHQ가 30배 우수
선택도 50%: NHQ가 10배 우수

선택도가 극단적일수록 NHQ의 이점이 더 크다는 것을 알 수 있다.

18.4 기술적 깊이: 알고리즘 상세

HNSW 복습

표준 HNSW 탐색 (검색 k개, top-k 답변):

1. entry point에서 시작
2. 층을 내려가며 coarse-grained 탐색 수행
3. bottom 층에서 세밀한 탐색 (greedy search with local optimization)
4. 후보를 거리 순서대로 정렬하여 top-k 반환

NHQ 탐색 (NPG 기반)

NHQ 탐색 (속성 조건 C 포함):

1. entry point에서 시작
2. 층을 내려가며 탐색
3. 각 층에서 후보 노드 평가:
 - 거리 조건: $d(q, \text{candidate}) < \text{threshold?}$
 - 속성 조건: $\text{candidate.attr} \in C?$
 - 둘 다 만족: 탐색 대상 추가
 - 거리만 만족: threshold 갱신 후 무시 (속성 불만족)
 - 속성만 만족: 탐색 대상 추가 (거리는 나중에 확인)

핵심 최적화:

정렬 우선순위 (후보 선택):

1. 거리가 가깝고, 속성도 만족하는 후보 (높은 우선순위)
2. 거리는 멀지만 속성만 만족 (중간)
3. 거리는 가깝지만 속성 불만족 (낮음 - 버려짐)
4. 둘 다 불만족 (매우 낮음 - 버려짐)

메모리 오버헤드 분석

표준 HNSW:

- 노드당 ~200 바이트 (벡터 + 메타데이터)

NPG-C:

- 노드당 ~250 바이트 (벡터 + 속성값 + 메타데이터)
- 크기: 필터링 후 데이터 크기에만 비례

NPG-A:

- 기본 그래프: ~200 바이트/노드 (전체 데이터)
- 속성 인덱스: 속성 종류와 데이터 크기에 비례
- 총 크기: 기본 그래프 + 인덱스 (대부분 무시 가능)

18.5 본 논문과의 관계

이 논문의 NHQ 프레임워크는 **단일 테이블 내의 속성 필터링**에 특화되어 있다. Exqutor와의 관계를 정리하면:

NHQ가 다루는 범위:

```
SELECT * FROM products
WHERE embedding <-> query_vector < threshold
AND category = 'electronics'
AND price > 100
```

Exqutor가 다루는 범위:

```
SELECT o.order_id, l.line_total
FROM orders o
JOIN lineitem l ON o.order_key = l.order_key
JOIN part p ON l.part_key = p.part_key
WHERE p.embedding <-> query_vector < threshold -- NHQ 영역
AND o.order_date > '2020-01-01'
AND p.price > 50
ORDER BY l.line_total DESC
LIMIT 10;
```

Exqutor 논문의 Related Work에서 "기존 필터링된 벡터 검색 연구([7], [18] 등)는 단일 테이블에 한정되며, 여러 테이블에 걸친 조인 쿼리에서의 최적화는 다루지 않는다"라고 명시한다.

두 접근법의 통합 가능성:

NHQ의 단일 테이블 최적화와 Exqutor의 멀티 테이블 최적화는 **이론적으로 결합 가능하다**:

- 1단계: NHQ를 각 벡터 필터가 있는 테이블에 적용
→ 개별 테이블의 선택도 추정 정확성 향상
- 2단계: Exqutor의 ECQ0로 조인 순서/방식 최적화
→ 전체 조인 계획의 효율성 극대화

이렇게 하면 계층적 최적화(hierarchical optimization)가 가능해진다:

- 단일 테이블 수준: NHQ의 벡터 필터 이점
- 멀티 테이블 수준: Exqutor의 조인 계획 이점

18.6 실무적 의미

NHQ가 보여주는 가장 중요한 통찰은 "속성과 벡터를 별도 문제로 보면 안 된다"는 것이다:

- **Pre-filtering**: 속성을 우선으로 봄 → "벡터는 부차적"
- **Post-filtering**: 벡터를 우선으로 봄 → "속성은 사후 필터"
- **NHQ**: 둘을 동등하게 봄 → "인덱스부터 통합설계"

이것은 Exqutor의 카디널리티 추정이 중요한 이유를 다시 한 번 강조한다: **정확한 선택도를 알면, 최적 전략(인덱스 구조, 탐색 알고리즘, 프루닝 방식)이 자동으로 결정된다.**

추가 제기 문제

1. 인덱스 구축 오버헤드

NPG-C는 각 속성값마다 별도 그래프를 구축해야 할 수도 있다. 데이터가 자주 업데이트되는 환경에서는:

- 새로운 데이터 삽입 시 NPG 그래프 재조정 비용
- 속성값의 추가/제거 시 그래프 수정 비용

실험은 정적 데이터셋에 기반하므로, 동적 업데이트 환경에서의 성능이 검증되지 않았다.

2. 복합 필터 확장성

논문에서 테스트한 속성은 대부분 **단일 속성 또는 2개 속성**이다. 실세계의 많은 쿼리는 5개 이상의 속성 조건을 갖는다:

```
WHERE category = 'electronics'
AND price > 100
AND availability = 'in_stock'
AND rating > 4.0
AND delivery_time < 3
```

이런 경우 NPG의 조인 조건 개수가 기하급수적으로 증가하는지, 성능이 선형적으로 유지되는지 불명확하다.

3. 고차원 벡터의 이점 감소

실험의 벡터 차원이 대부분 128D~4096D이다. 최근 대형 언어모델의 임베딩은 1536D~3072D, 또는 그 이상이다. 고차원에서 속성 필터의 이점이 유지되는지 확인 필요.

추가 제기 문제

1. 인덱스 구축 및 유지 비용의 상세 분석 부족

NPG-C와 NPG-A 모두 구축 비용을 명시적으로 분석하지 않는다. 특히:

- 속성값 분포가 심하게 불균형할 때 (한 속성값에 99% 데이터), NPG-C 그래프들의 크기 차이
- 데이터 업데이트 시 그래프 재조정 비용

이런 비용이 무시할 수 없을 정도로 커질 수 있는데, 정적 데이터 환경 가정이 한계.

2. 다중 속성 조건으로의 확장 한계

현재 알고리즘은 단일 속성 또는 소수 속성 조건에 최적화되어 있다. 실제 분석 쿼리는 5개 이상의 속성 필터를 자주 갖는다.

이 경우:

- Joint pruning의 효율성 감소 (조건이 많을수록 프루닝 가능성 높음 → 탐색 범위 격감)
- 메모리 오버헤드 증가

3. 속성 선택도의 동적 변화 대응

실험에서는 고정 선택도를 가정하지만, 실제 워크로드는 선택도가 쿼리마다 크게 변한다. 최적 전략(NPG-C vs NPG-A)을 동적으로 선택하는 메커니즘이 필요한데, 오버헤드와 정확성의 균형이 모호함.

4. 벡터 품질과 속성 분포의 상관성

논문은 벡터와 속성이 독립적이라고 가정하지만, 실세계에서는 상관성이 있을 수 있다. 예: "고급 카메라"는 특정 이미지 특징을 자주 보임. 이 경우 joint pruning의 효율성이 달라질 수 있음.
